BookLibrary API

A Complete Student Reference Guide

FastAPI · SQLModel · JWT Auth · Async Python · Docker

Everything you need to understand this project — from first principles to production patterns.

Table of Contents

1. Project Overview & Architecture
2. Project Structure
3. Configuration & Environment (config.py, .env)
4. Database Layer (database.py)
5. Models — SQLModel Tables (book.py, users.py)
6. Schemas — Pydantic Validation (schemas/book.py, schemas/user.py)
7. CRUD Operations (crud/book.py)
8. Security: Passwords & JWT (security.py)
9. Authentication Router (auth.py)
10. Dependencies & Guards (dependencies.py)
11. Books Router (books.py)
12. Middleware (middleware.py)
13. Application Entry Point (main.py)
14. Database Migrations with Alembic (alembic/env.py)
15. Testing (tests/test_books.py)
16. Docker & Deployment
17. Key Concepts Quick Reference
18. Practice Project Ideas

1. Project Overview & Architecture

This project is a RESTful API for a digital book library, built with FastAPI. It supports user registration, JWT-based authentication, role-based access control (users vs admins), and full CRUD management of books — including file uploads for cover images.

What is FastAPI?

FastAPI is a modern Python web framework designed for building APIs quickly and correctly. Its key features are:

- Automatic documentation (Swagger UI at /api/v1/docs, ReDoc at /api/v1/redoc)
- Built-in data validation using Pydantic models
- Async-first: supports Python's async/await natively
- Type hints everywhere — Python knows the shape of your data at every step

Request Lifecycle — How a Request Flows Through the App

Request Flow (each incoming HTTP request travels this path)
1. Browser / client sends HTTP request (e.g. GET /api/v1/books/)
2. CORS Middleware — checks allowed origins
3. RequestTimingMiddleware — records start time
4. FastAPI Router — matches URL to a route function (endpoint)
5. Dependency Injection — pagination params, DB session, auth token extracted
6. Route Function — calls CRUD functions with the DB session
7. CRUD Function — runs SQL via SQLModel against SQLite
8. Response — Pydantic schema serializes the data back to JSON
9. RequestTimingMiddleware — adds X-Process-Time header
10. Client receives JSON response

Technology Stack

Technology	Purpose
FastAPI	Web framework — routing, validation, docs
SQLModel	ORM — maps Python classes to database tables
SQLite + aiosqlite	Database — file-based, zero setup, async-ready
Pydantic	Data validation & serialization (baked into FastAPI)
python-jose	JWT creation and verification
bcrypt	Secure password hashing
Alembic	Database migration management
pytest	Automated testing
Docker	Containerisation for consistent deployment

2. Project Structure

Understanding where each file lives and why is the first step to navigating the codebase confidently.

```
BookLibary/
├── app/
│   ├── core/
│   │   ├── database.py      ← async engine, session factory, sync table creator
│   │   ├── security.py      ← bcrypt hashing + JWT sign/verify
│   │   └── middleware.py    ← request timing middleware
│   ├── models/
│   │   ├── book.py          ← Book database table (SQLModel)
│   │   └── users.py         ← User database table (SQLModel)
│   ├── schemas/
│   │   ├── book.py          ← BookCreate / BookUpdate / BookRead (Pydantic)
│   │   └── user.py          ← UserCreate / UserRead (Pydantic)
│   ├── crud/
│   │   └── book.py          ← DB operations: create, read, update, delete
│   ├── routers/
│   │   ├── auth.py          ← /register and /token endpoints
│   │   └── books.py         ← /books CRUD endpoints with auth guards
│   ├── dependencies.py      ← reusable FastAPI dependencies
│   ├── config.py            ← app settings from .env
│   └── main.py              ← app entry point
├── alembic/
│   └── env.py               ← migration runner config
├── tests/
│   └── test_books.py        ← pytest test suite
├── static/covers/           ← uploaded book cover images
├── .env                     ← secret config (never commit!)
```

```
├── Dockerfile
└── docker-compose.yml
```

Key Design Pattern: Separation of Concerns
Each layer has one job:
models/ → define the database shape (tables, columns, types)
schemas/ → define what data is accepted IN and sent OUT via the API
crud/ → talk to the database (SQL queries)
routers/ → handle HTTP requests and responses
core/ → infrastructure (DB connection, security, middleware)
Why? If your database changes, you only touch models/ and crud/.
If your API shape changes, you only touch schemas/ and routers/.
Bugs are isolated. Changes are safe.

3. Configuration & Environment

Files: `app/config.py` and `.env`

Why Separate Config from Code?

Hardcoding values like SECRET_KEY or DATABASE_URL directly in your code is dangerous and inflexible. If you ever push that code to GitHub, your secrets are exposed. If you need different settings for development vs production, you'd have to change the code itself.

The solution is the 12-Factor App principle: store configuration in environment variables.

How It Works

```
# config.py

from pydantic_settings import BaseSettings, SettingsConfigDict



class Settings(BaseSettings):

    model_config = SettingsConfigDict(env_file='.env', extra='ignore')



    APP_NAME: str = 'Book Library API'

    SECRET_KEY: str              # No default — MUST be set in .env

    DATABASE_URL: str = 'sqlite+aiosqlite:///booklibary.db'



settings = Settings()   # Singleton — imported everywhere
```

pydantic-settings reads the .env file automatically and maps variables to Python attributes with type validation. If SECRET_KEY is missing from .env, the app refuses to start — preventing a silent security failure.

Your .env file should look like this (never commit it to Git!)
SECRET_KEY=your-very-long-random-secret-key-here-at-least-32-chars
DATABASE_URL=sqlite+aiosqlite:///booklibary.db

The Singleton Pattern

Notice settings = Settings() is called once at module level. Every other file imports that one instance:

```
from app.config import settings



print(settings.SECRET_KEY)    # Same object every time

print(settings.DATABASE_URL)
```

This means settings are loaded once at startup. It's efficient and consistent.

4. Database Layer

File: `app/core/database.py`

Synchronous vs Asynchronous Database Access

Traditional database drivers block the entire Python process while waiting for a query to complete. This is fine for simple scripts but terrible for web servers handling many concurrent users.

Synchronous (blocking)	Asynchronous (non-blocking)
Thread waits for DB	Thread is freed while DB works
1 thread = 1 request	1 thread can handle many requests
Uses sqlite3 / psycopg2	Uses aiosqlite / asyncpg
Simple to understand	Requires async/await syntax

The Async Setup

```
# database.py

from sqlmodel.ext.asyncio.session import AsyncSession

from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker


async_engine = create_async_engine(settings.DATABASE_URL, echo=False)


AsyncSessionLocal = async_sessionmaker(

    async_engine, class_=AsyncSession, expire_on_commit=False

)


async def get_async_session() -> AsyncGenerator[AsyncSession, None]:

    async with AsyncSessionLocal() as session:

        yield session
```

Understanding async def and yield

get_async_session is a generator function (note yield). FastAPI uses it as a dependency: it opens a session, yields it to your route function, then closes it when the request is done — even if an error occurred. This pattern is called a context manager.

What is a DB Session?
Think of a session as a conversation with the database:
- You open a session (start talking)
- You run queries (exchange messages)
- You commit (save your changes permanently)
- You close the session (end the conversation)
Using 'async with' ensures the session is always closed properly,
even if your code crashes midway through.

The Sync Fallback for Startup

```
def create_db_and_tables_sync():

    sync_url = settings.DATABASE_URL.replace('sqlite+aiosqlite', 'sqlite', 1)

    engine = create_engine(sync_url)

    SQLModel.metadata.create_all(engine)
```

SQLModel.metadata.create_all() reads all imported model classes and creates any missing tables. It's called synchronously at startup because the async engine isn't available yet when the app first boots.

5. Models — SQLModel Tables

Files: `app/models/book.py` and `app/models/users.py`

What is an ORM?

An ORM (Object-Relational Mapper) lets you work with database tables as Python classes. Instead of writing raw SQL like SELECT * FROM book WHERE id = 5, you write Python code that looks like any other code.

The Book Model

```
# models/book.py

from sqlmodel import SQLModel, Field

from typing import Optional


class Book(SQLModel, table=True):

    id: Optional[int] = Field(default=None, primary_key=True)

    title: str

    author: str

    isbn: str = Field(unique=True)       # No two books share an ISBN

    description: Optional[str] = None    # Nullable column

    published_year: int

    cover_image: Optional[str] = None    # Stores file path as text
```

Anatomy of a SQLModel Field
id: Optional[int] = Field(default=None, primary_key=True)
Optional[int] → column can be NULL in the DB (Python: can be None)
default=None → Python default before saving to DB
primary_key=True → unique identifier for every row; auto-increments

isbn: str = Field(unique=True)
str → NOT NULL column (required)
unique=True → database enforces no duplicates at the DB level

The User Model

```
# models/users.py

class User(SQLModel, table=True):

    id: Optional[int] = Field(default=None, primary_key=True)

    username: str = Field(unique=True, index=True)   # index speeds up lookups

    email: str = Field(unique=True, index=True)

    hashed_password: str    # NEVER store plain text passwords

    role: str = Field(default='user')   # 'user' or 'admin'

    is_active: bool = True

    created_at: datetime = Field(default_factory=datetime.utcnow)
```

Concept	Why It Matters
hashed_password (not password)	Passwords are hashed before storage — even if the DB leaks, passwords are safe
index=True on username/email	Creates a B-tree index; lookups go from O(n) to O(log n) — much faster
role field	Enables role-based access control (RBAC) without a separate permissions table
is_active flag	Soft-disable accounts without deleting them — preserves history
table=True	Tells SQLModel this is a real DB table, not just a schema class

6. Schemas — Pydantic Validation

Files: `app/schemas/book.py` and `app/schemas/user.py`

Models vs Schemas — What's the Difference?

SQLModel (models/)	Pydantic (schemas/)
Represents a DB table	Represents API input/output shapes
Has table=True	No table — pure data validation
Has ALL columns incl. hashed_password	Exposes only safe fields to clients
Used by CRUD functions	Used by route function signatures
One class per table	Multiple per concept: Create, Update, Read

Book Schemas

```
# schemas/book.py

class BookBase(BaseModel):

    title: str = Field(..., min_length=1, max_length=200)

    author: str = Field(..., min_length=1, max_length=100)

    isbn: str = Field(..., min_length=10, max_length=13)

    description: Optional[str] = Field(None, max_length=1000)

    published_year: int = Field(..., ge=1500, le=2026)


class BookCreate(BookBase): pass      # Inherits all, no extra fields


class BookUpdate(BaseModel):          # All fields optional for partial update

    title: Optional[str] = Field(None, min_length=1, max_length=200)

    # ... same fields but all Optional


class BookRead(BookBase):             # What clients receive back
```

```
    id: int                          # Adds DB-generated id

    cover_image: Optional[str] = None # Adds server-generated field
```

Field Validators — Your First Line of Defence
Field(..., min_length=1) → '...' means required. Empty strings rejected.
Field(..., max_length=200) → Prevents database overflow attacks.
Field(..., ge=1500) → ge = 'greater than or equal'. No ancient books!
Field(..., le=2026) → le = 'less than or equal'. No future books!
FastAPI returns a 422 Unprocessable Entity error automatically if
any of these constraints are violated — before your code even runs.

Why Three Book Schemas?

- BookCreate — used when adding a book: all required, no id (DB generates it)
- BookUpdate — used when editing: all optional so you can change just the title
- BookRead — used in responses: includes id and cover_image, which only exist after DB save

7. CRUD Operations

File: `app/crud/book.py`

What is CRUD?

CRUD stands for Create, Read, Update, Delete — the four fundamental operations of any data-driven application. Keeping these in a dedicated crud/ folder means your route functions stay thin: they only handle HTTP concerns, not database logic.

Create

```
async def create_book(session: AsyncSession, book_in: BookCreate) -> Book:

    db_book = Book.model_validate(book_in)  # Schema → Model conversion

    session.add(db_book)

    await session.commit()       # Writes to disk

    await session.refresh(db_book)  # Reloads from DB (gets auto-generated id)

    return db_book
```

model_validate() — The Bridge Between Schema and Model
Book.model_validate(book_in) converts a BookCreate Pydantic object into a
Book SQLModel object. It copies matching field names automatically.
After session.commit(), the 'id' field is populated by SQLite's autoincrement.
session.refresh(db_book) fetches that updated row back into the Python object
so you can return it with the id included.

Read with Filtering

```
async def get_all_books(session, skip=0, limit=10,

                        title=None, author=None, ...) -> List[Book]:
```

```
    statement = select(Book)


    if title:
        statement = statement.where(Book.title.ilike(f'%{title}%'))
    if author:
        statement = statement.where(Book.author.ilike(f'%{author}%'))
    if min_year:
        statement = statement.where(Book.published_year >= min_year)


    statement = statement.offset(skip).limit(limit)
    result = await session.exec(statement)
    return result.all()
```

SQL Concept	Code Equivalent
SELECT * FROM book	select(Book)
WHERE title LIKE '%python%'	Book.title.ilike('%python%') (case-insensitive)
OFFSET 20	.offset(20) — skip first 20 rows (pagination)
LIMIT 10	.limit(10) — return at most 10 rows
WHERE published_year >= 2000	Book.published_year >= 2000

Update (Partial)

```
async def update_book(session, book_id, book_in: BookCreate):
    db_book = await get_book_by_id(session, book_id)
    if not db_book:
        return None
    for key, value in book_in.model_dump(exclude_unset=True).items():
        setattr(db_book, key, value)
```

```
    session.add(db_book)

    await session.commit()

    await session.refresh(db_book)

    return db_book
```

The key is `exclude_unset=True`. Without it, model_dump() returns None for every optional field not provided, and you'd accidentally overwrite existing data with nulls. With it, only the fields the client actually sent are updated.

8. Security: Passwords & JWT

File: `app/core/security.py`

Password Hashing with bcrypt

Never store passwords as plain text. If your database is compromised, attackers would have every user's password immediately. Instead, we store a hash — a one-way transformation.

```
import bcrypt


def get_password_hash(password: str) -> str:
    salt = bcrypt.gensalt()   # Random bytes added before hashing
    return bcrypt.hashpw(password.encode('utf-8'), salt).decode('utf-8')


def verify_password(plain_password: str, hashed_password: str) -> bool:
    return bcrypt.checkpw(plain_password.encode('utf-8'),
                          hashed_password.encode('utf-8'))
```

Why bcrypt? Why not MD5 or SHA-256?
MD5/SHA-256 are fast — attackers can try billions of guesses per second.
bcrypt is intentionally slow (configurable 'work factor'). It also automatically
adds a random 'salt' to prevent rainbow table attacks — two identical passwords
produce different hashes every time.
Example:
password: 'hello123'
hash1: $2b$12$abc... (different every time)

hash2: $2b$12$xyz... (same password, different salt = different hash)
verify_password() handles this: it extracts the salt from the stored hash
and re-applies it to the input before comparing.

JSON Web Tokens (JWT)

After a user logs in, we need a way to identify them on future requests without asking for their password every time. JWTs solve this.

```
def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(minutes=30)
    to_encode.update({'exp': expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm='HS256')


def decode_access_token(token: str):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=['HS256'])
        return payload
    except JWTError:
        raise HTTPException(status_code=401, ...)
```

JWT Structure — Three Base64-encoded Parts		
eyJhbGciOiJIUzI1NiJ9 .	eyJzdWIiOiJhbGljZSIsImV4cCI6MTcwMH0 .	<signature>
HEADER	PAYLOAD	SIGNATURE
Header: algorithm used (HS256)		

Payload: your data — { 'sub': 'alice', 'exp': 1234567890 }
Signature: HMAC(header + payload, SECRET_KEY) — tamper-proof
Anyone can READ the header and payload (they're just base64-encoded).
But only the server with SECRET_KEY can VERIFY the signature.
If an attacker modifies the payload, the signature check fails.

Term	Meaning
sub (subject)	Who the token is about — typically the username
exp (expiry)	Unix timestamp after which the token is invalid
HS256	HMAC-SHA256 — symmetric signing algorithm (same key to sign and verify)
Bearer token	HTTP convention: sent as Authorization: Bearer <token> header

9. Authentication Router

File: `app/routers/auth.py`

Two Endpoints

Endpoint	What It Does
POST /api/v1/auth/register	Creates a new user account. Hashes password. Returns username/email/role.
POST /api/v1/auth/token	Validates credentials. Returns a JWT access token + token_type + role.

Register Flow

```
@router.post('/register', response_model=dict, status_code=201)

async def register_user(user_in: UserCreate, session =
Depends(get_async_session)):

    # 1. Check for duplicate username or email

    statement = select(User).where(

        (User.username == user_in.username) | (User.email == user_in.email)

    )

    existing_user = (await session.exec(statement)).first()

    if existing_user:

        raise HTTPException(400, 'Username or email already registered')



    # 2. Hash the password

    hashed_password = get_password_hash(user_in.password)



    # 3. Save user to DB

    user = User(username=user_in.username, email=user_in.email,

                hashed_password=hashed_password, role=user_in.role)

    session.add(user)
```

```
    await session.commit()
```

Login Flow (OAuth2 Password Flow)

```
@router.post('/token')

async def login_for_access_token(

    form_data: OAuth2PasswordRequestForm = Depends(),   # username + password form

    session = Depends(get_async_session)

):

    # 1. Find user by username

    user = (await session.exec(select(User).where(User.username ==
form_data.username))).first()



    # 2. Verify password hash

    if not user or not verify_password(form_data.password, user.hashed_password):

        raise HTTPException(401, 'Incorrect username or password')



    # 3. Issue JWT

    access_token = create_access_token(data={'sub': user.username})

    return {'access_token': access_token, 'token_type': 'bearer', 'role':
user.role}
```

Why does login use a Form, not JSON?
OAuth2PasswordRequestForm is a standard. It expects a multipart/form-data
body with 'username' and 'password' fields.
This is why Swagger UI (/api/v1/docs) shows a login form with two input boxes
rather than a JSON body editor for the /token endpoint.

The OAuth2 standard chose forms because it predates widespread JSON API usage.

10. Dependencies & Guards

File: `app/dependencies.py`

What is Dependency Injection?

FastAPI's Depends() system lets you declare reusable logic that runs before your route function. Think of it as composable middleware for individual endpoints.

```
# Instead of repeating this in every books route:

async def some_route(session: AsyncSession = Depends(get_async_session)):

    ...



# You can compose dependencies:

async def create_book_endpoint(

    pagination_params: dict = Depends(pagination),

    filters: dict = Depends(book_filters),

    session: AsyncSession = Depends(get_async_session),

    current_admin = Depends(get_current_admin),  # Auth guard

):

    ...
```

Pagination Dependency

```
def pagination(

    skip: int = Query(0, ge=0),

    limit: int = Query(10, ge=1, le=100)

) -> Dict[str, int]:

    return {'skip': skip, 'limit': limit}
```

Query() tells FastAPI these come from the URL query string: GET /books/?skip=20&limit=5. The ge/le constraints validate them automatically.

Authentication Guard Chain

How get_current_admin works — a dependency chain
get_current_admin → depends on → get_current_user
↓
1. Extracts Bearer token from Authorization header
2. Calls decode_access_token(token)
3. Gets 'sub' (username) from JWT payload
4. Fetches User from DB
5. Checks user.is_active
6. Returns User object
↓
get_current_admin then checks: if user.role != 'admin' → raise 403 Forbidden

```
async def get_current_user(

    token: str = Depends(oauth2_scheme),   # Extracts from header automatically

    session: AsyncSession = Depends(get_async_session)

) -> User:

    payload = decode_access_token(token)   # Raises 401 if invalid/expired

    username = payload.get('sub')

    user = (await session.exec(select(User).where(User.username ==
username))).first()

    if not user or not user.is_active:

        raise HTTPException(401, 'User not found or inactive')

    return user



async def get_current_admin(current_user: User = Depends(get_current_user)):
```

```
    if current_user.role != 'admin':
        raise HTTPException(403, 'Admin access required')
    return current_user
```

11. Books Router

File: `app/routers/books.py`

Route Summary

Method + Path	Auth Required	What It Does
GET /books/	None (public)	List all books with filtering & pagination
GET /books/{book_id}	None (public)	Get one book by ID
POST /books/	Admin only	Create a book with cover image upload
PUT /books/{book_id}	Admin only	Update a book's details
DELETE /books/{book_id}	Admin only	Delete a book

File Upload — multipart/form-data

```
@router.post('/')
async def create_book_endpoint(
    background_tasks: BackgroundTasks,
    session = Depends(get_async_session),
    current_admin = Depends(get_current_admin),
    title: str = Form(..., min_length=1, max_length=200),
    author: str = Form(...),
    isbn: str = Form(...),
    published_year: int = Form(..., ge=1500),
    cover: UploadFile = File(...),
):
    file_extension = cover.filename.split('.')[-1].lower()
    filename = f'{isbn}.{file_extension}'
    with open(f'static/covers/{filename}', 'wb') as buffer:
        shutil.copyfileobj(cover.file, buffer)
```

```
    # Then create book record with cover_image path...
```

Why Form() instead of a JSON body here?
File uploads use multipart/form-data encoding, not JSON.
You cannot mix a JSON body and file upload in one request.
FastAPI's Form() and File() extract data from multipart fields.
This means the POST /books/ endpoint requires multipart form data,
not a JSON body — which you can test in Swagger UI.

Background Tasks

```
async def log_action(message: str):
    print(f'[BACKGROUND LOG] {message}')


# In the route:
background_tasks.add_task(log_action, f'New book created: {title}')
```

BackgroundTasks lets you run code after the HTTP response is already sent. This is useful for logging, sending emails, or updating caches — tasks that don't need to block the client waiting.

lru_cache — In-Memory Caching

```
@lru_cache(maxsize=200)
def _cache_key(skip, limit, title, author, published_year, min_year, max_year):
    return (skip, limit, title, author, published_year, min_year, max_year)
```

`lru_cache` (Least Recently Used cache) stores function results in memory. For identical inputs, it returns the cached result instantly. With maxsize=200, it remembers the 200 most recent unique query combinations before evicting old ones. Note: this is a simple demo pattern — in production, use Redis for distributed caching.

12. Middleware

File: `app/core/middleware.py`

What is Middleware?

Middleware wraps every request and response passing through your app. Think of it as a decorator for the entire application — code that runs before every route function and after every response.

```
from starlette.middleware.base import BaseHTTPMiddleware


class RequestTimingMiddleware(BaseHTTPMiddleware):

    async def dispatch(self, request: Request, call_next):

        start_time = time.time()

        response = await call_next(request)   # Run the actual route

        process_time = time.time() - start_time

        response.headers['X-Process-Time'] = str(process_time)

        print(f'{request.method} {request.url.path} - {process_time:.4f}s')

        return response
```

How call_next works
dispatch() is called for every request. call_next(request) passes the
request to the next layer (the actual route handler) and returns the response.
This creates a sandwich pattern:
→ before call_next: pre-processing (e.g. record start time, auth check)
→ call_next(request): route runs, response generated
→ after call_next: post-processing (e.g. add headers, log timing)

The X-Process-Time header is visible in browser DevTools → Network tab.

CORS Middleware

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],        # Allow all origins (dev only!)
    allow_credentials=True,
    allow_methods=['*'],        # GET, POST, PUT, DELETE...
    allow_headers=['*'],
)
```

CORS (Cross-Origin Resource Sharing) is a browser security feature. By default, a browser blocks JavaScript on localhost:3000 from calling an API on localhost:8000. CORSMiddleware adds the right HTTP headers to allow this. In production, replace '*' with your specific frontend domain.

13. Application Entry Point

File: `app/main.py`

Lifespan Context Manager

```
@asynccontextmanager

async def lifespan(app: FastAPI):

    # ── STARTUP ──────────────────────────────

    os.makedirs('static/covers', exist_ok=True)   # Ensure upload dir exists

    create_db_and_tables_sync()                   # Create DB tables

    yield

    # ── SHUTDOWN ─────────────────────────────

    # (cleanup code would go here if needed)
```

The lifespan pattern replaces the deprecated @app.on_event('startup') decorator. Everything before yield runs on startup; everything after runs on shutdown. This is more testable and composable.

App Configuration

```
app = FastAPI(

    title=settings.APP_NAME,

    description='Learn FastAPI by building a BookLibrary API',

    version='1.0.0',

    openapi_url='/api/v1/openapi.json',

    docs_url='/api/v1/docs',        # Swagger UI

    redoc_url='/api/v1/redoc',      # ReDoc

    lifespan=lifespan,

)
```

Global Exception Handler

```
@app.exception_handler(Exception)

async def global_exception_handler(request: Request, exc: Exception):

    print(f'Internal Error: {exc}')

    return JSONResponse(status_code=500,

                        content={'detail': 'Internal server error'})
```

Without this, unhandled exceptions would bubble up and expose Python stack traces to clients — a security risk. This handler catches everything, logs it server-side, and returns a safe generic error.

Router Registration

```
app.include_router(auth_router, prefix='/api/v1')

app.include_router(books_router, prefix='/api/v1')

# auth_router has prefix='/auth'   → final: /api/v1/auth/register

# books_router has prefix='/books' → final: /api/v1/books/
```

14. Database Migrations with Alembic

File: `alembic/env.py`

Why Do We Need Migrations?

SQLModel.metadata.create_all() creates tables that don't exist yet — but it cannot ALTER existing tables. If you add a column to your Book model after the database already exists, create_all() does nothing. Your DB is now out of sync with your code.

Alembic solves this with versioned migration scripts
Each migration is a Python file with upgrade() and downgrade() functions.
upgrade(): applies the change (e.g. adds a new column)
downgrade(): reverses it (e.g. drops that column)
Alembic tracks which migrations have been applied in a special
'alembic_version' table in your database.

The env.py Configuration

```
# alembic/env.py

config.set_main_option('sqlalchemy.url', settings.DATABASE_URL)

target_metadata = SQLModel.metadata   # Points to your models



# Models must be imported here so metadata is populated:

from app.models.book import Book
```

Common Alembic Commands

Command	What It Does
alembic revision --autogenerate -m 'add genre column'	Generate a new migration by comparing models to DB
alembic upgrade head	Apply all pending migrations
alembic downgrade -1	Undo the last migration
alembic history	Show all migration versions
alembic current	Show which migration the DB is currently at

Important: Always review autogenerated migrations before applying!
Alembic's --autogenerate is smart but not perfect.
It may miss: table renames, custom constraints, or data migrations.
Open the generated file in alembic/versions/ and verify it looks correct
before running 'alembic upgrade head'.

15. Testing

File: `tests/test_books.py`

Testing Strategy

The test suite uses pytest and FastAPI's TestClient — a synchronous HTTP client that talks to your app in-process without needing a real server running.

```
# tests/test_books.py

TEST_DATABASE_URL = 'sqlite:///./test.db'  # Separate test database!

engine = create_engine(TEST_DATABASE_URL)



def override_get_session():

    with Session(engine) as session:

        yield session



@pytest.fixture(scope='session', autouse=True)

def setup_test_db():

    SQLModel.metadata.create_all(engine)   # Create tables

    yield

    SQLModel.metadata.drop_all(engine)     # Clean up after all tests



@pytest.fixture

def client():

    app.dependency_overrides[get_async_session] = override_get_session

    with TestClient(app) as c:

        yield c

    app.dependency_overrides.clear()
```

Key Testing Concepts

Concept	Explanation
Fixtures	Functions decorated with @pytest.fixture that set up test state. Tests declare them as parameters.
scope='session'	The fixture runs once for the entire test session (not once per test). Used for expensive setup like creating DB tables.
dependency_overrides	FastAPI lets you replace any Depends() with a test version. Here we swap the real async DB session for a synchronous test one.
autouse=True	The fixture runs automatically for every test without being explicitly requested.

The Tests

```
def test_get_all_books(client):

    response = client.get('/api/v1/books/')

    assert response.status_code == 200

    assert isinstance(response.json(), list)   # Must be an array



def test_create_book_unauthorized(client):

    response = client.post('/api/v1/books/', json={})

    assert response.status_code in [401, 422]

    # 401 = no auth token provided

    # 422 = validation error (missing required fields)

    # Both are valid 'rejection' responses for an unauthorised request
```

Run the tests with:
pytest tests/ # Run all tests
pytest tests/ -v # Verbose: show each test name
pytest tests/ -v -s # Also show print() output
pytest tests/test_books.py::test_get_all_books # Run one test

16. Docker & Deployment

Why Docker?

'It works on my machine' is a classic developer problem. Docker packages your app and all its dependencies into a container — a lightweight, isolated environment that runs identically on any machine, from a developer's laptop to a cloud server.

Dockerfile

```
FROM python:3.12-slim


WORKDIR /app


COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt


COPY . .


RUN mkdir -p static/covers


CMD ['uvicorn', 'app.main:app', '--host', '0.0.0.0', '--port', '8000']
```

docker-compose.yml

```
version: '3.8'

services:

  web:

    build: .

    ports:

      - '8000:8000'
```

```
    volumes:
      - ./static:/app/static    # Persist uploaded files
    env_file:
      - .env                    # Inject secrets from .env
```

Docker Concept	Explanation
FROM python:3.12-slim	Base image — starts from an official Python 3.12 image (slim = smaller size)
WORKDIR /app	All subsequent commands run inside /app in the container
COPY requirements.txt first	Docker caches layers — copying requirements before code means pip install only re-runs when dependencies change, not every code edit
--host 0.0.0.0	Listen on all interfaces inside the container; required so port mapping works
volumes:	Mounts a host directory into the container — uploads survive container restarts

Quick Docker Commands
docker compose up --build # Build image and start containers
docker compose up -d # Start in background (detached)
docker compose down # Stop and remove containers
docker compose logs -f # Follow live logs
docker compose exec web bash # Open shell inside running container

17. Key Concepts Quick Reference

HTTP Status Codes Used in This Project

Code	Name	When Used
200	OK	Successful GET / PUT
201	Created	Successful POST (resource created)
204	No Content	Successful DELETE (no body to return)
400	Bad Request	Duplicate user, inactive user
401	Unauthorized	Missing or invalid JWT token
403	Forbidden	Valid token but not admin
404	Not Found	Book or user doesn't exist in DB
422	Unprocessable Entity	Pydantic validation failed
500	Internal Server Error	Unhandled exception

async / await Cheat Sheet

```
# 'async def' marks a function as a coroutine

async def my_func():

    # 'await' suspends this function until the awaited task completes

    # while OTHER requests can be handled

    result = await some_db_query()

    return result



# You can only 'await' inside an 'async def'

# FastAPI runs your async route functions inside an event loop automatically
```

Common Mistakes & Fixes

Mistake	Fix
Forgetting await before DB calls	All session.exec(), session.commit(), session.refresh() must be awaited
Returning the model directly from a route that expects a schema	response_model= in @router.get() controls serialisation — make sure it matches
Committing without refreshing	After commit(), the id is set by the DB. Call refresh() to get it back in Python
Using model_dump() without exclude_unset=True for updates	You'll overwrite existing data with None for fields not sent by the client
SECRET_KEY not in .env	The app will crash at startup with a validation error — intentional by design
cover_image stored with absolute path	Store relative paths like /covers/isbn.jpg; prefix with your domain in the frontend

Glossary

Term	Definition
ORM	Object-Relational Mapper — maps Python classes to DB tables
Pydantic	Python library for data validation using type annotations
SQLModel	Library combining SQLAlchemy (ORM) + Pydantic into one class
JWT	JSON Web Token — signed, self-contained token for auth
bcrypt	Password hashing algorithm designed to be slow (secure)
Dependency Injection	FastAPI's Depends() — reusable logic injected into routes
Middleware	Code that wraps every request/response in the app
Lifespan	Context manager for app startup and shutdown code
Migration	Versioned change to the DB schema (Alembic manages these)
CORS	Browser policy; CORSMiddleware adds headers to allow cross-origin requests
Bearer Token	Auth header convention: Authorization: Bearer <jwt>
LRU Cache	Least Recently Used — in-memory cache that evicts oldest entries
Background Task	Code that runs after the HTTP response is already sent
UploadFile	FastAPI type for handling multipart file uploads

18. Practice Project Ideas

The best way to solidify what you learned here is to build something yourself. Each project below is scoped so it reuses the exact same patterns from BookLibrary — models, schemas, CRUD, auth, file uploads — but in a new domain. Start from the one that interests you most.

Project 1 — Recipe Manager API

Build an API where users can store, browse, and manage cooking recipes. Admins can create/edit recipes; regular users can browse and save favourites.

Models to build

- Recipe — title, description, prep_time_minutes (int), servings (int), difficulty (str: easy/medium/hard), image (optional)
- Ingredient — name, quantity (str), unit (str), recipe_id (FK to Recipe)
- User — reuse the same User model from BookLibrary

Endpoints to implement

1. GET /recipes/ — list with filters: difficulty, max_prep_time, title search
2. GET /recipes/{id} — single recipe with its ingredients
3. POST /recipes/ — admin only, multipart form with image upload
4. POST /recipes/{id}/ingredients — admin only, add an ingredient to a recipe
5. DELETE /recipes/{id}/ingredients/{ing_id} — admin only
6. PUT /recipes/{id} — admin only, partial update
7. DELETE /recipes/{id} — admin only

New concepts you will practice

- One-to-many relationship: one Recipe has many Ingredients
- SQLModel relationship: `ingredients: List[Ingredient] = Relationship(back_populates='recipe')`
- Nested response schema: BookRead returned just a flat object; here RecipeRead should embed a list of IngredientRead objects
- Enum validation: use Python's Enum class to restrict difficulty to exactly three values

Stretch goals
Add a GET /recipes/random endpoint that returns a random recipe.
Add a favourite system: users can POST /recipes/{id}/favourite and GET /me/favourites.
Add a rating system: users POST /recipes/{id}/rating with a score 1-5,
and the recipe's average rating is computed and returned in RecipeRead.

Project 2 — Task Manager API (To-Do)

A personal task management API where each user sees only their own tasks. No shared admin model — every authenticated user can create, update, and delete their own data.

Models to build

- Task — title, description (optional), status (str: todo/in_progress/done), priority (int 1-5), due_date (date, optional), owner_id (FK to User)
- Tag — name, color (hex string), owner_id (FK to User)
- TaskTag — join table linking tasks to tags (many-to-many)

Endpoints to implement

8. GET /tasks/ — returns only the current user's tasks, filterable by status and priority
9. POST /tasks/ — any authenticated user creates a task assigned to themselves
10. PUT /tasks/{id} — user can only update their own task; return 403 if they try another user's
11. DELETE /tasks/{id} — same ownership check
12. POST /tasks/{id}/tags/{tag_id} — attach a tag to a task
13. GET /tags/ — list the current user's tags
14. POST /tags/ — create a tag

New concepts you will practice

- Ownership checks inside route functions — not just admin vs user, but 'does this record belong to the requesting user?'
- Many-to-many relationships via a join table (TaskTag)
- Date fields — FastAPI + Pydantic handle Python's date type natively
- Filtering by enum-like string status: `statement.where(Task.status == status)`

Stretch goals
Add GET /tasks/overdue — returns tasks where due_date < today and status != done.
Add a PATCH /tasks/{id}/status endpoint that only updates the status field.
Add pagination + sorting: ?sort_by=due_date&order=asc.

Project 3 — Movie Review API

An API where admins manage a catalogue of movies, and any registered user can write reviews. This introduces a true multi-user content system.

Models to build

- Movie — title, director, release_year, genre (str), poster_image (optional), avg_rating (float, computed)
- Review — content (str, max 2000 chars), rating (int 1-10), movie_id (FK), author_id (FK to User), created_at

Endpoints to implement

15. GET /movies/ — public, filterable by genre and min_rating
16. GET /movies/{id} — public, includes the 5 most recent reviews
17. POST /movies/ — admin only, with poster upload
18. POST /movies/{id}/reviews — any authenticated user, one review per user per movie
19. PUT /reviews/{id} — only the review author can edit their own review
20. DELETE /reviews/{id} — author OR admin can delete
21. GET /users/me/reviews — current user's review history

New concepts you will practice

- Computing a derived field (avg_rating) — either store and update it on every review write, or compute it with a SQL aggregate at read time
- Dual-role delete: the permission check needs to allow both the author and any admin
- Preventing duplicate reviews: query for an existing review by (movie_id, author_id) before inserting
- Nested schemas: MovieRead embeds a List[ReviewRead] for the recent reviews

Stretch goals
Add GET /movies/top-rated — returns movies sorted by avg_rating descending.
Add GET /genres — returns distinct genre values from the DB.
Add a helpful-votes system: users can mark a review as helpful (many-to-many).

Project 4 — Personal Finance Tracker API

A private API for tracking income and expenses. Every user manages only their own financial data. No admin role needed — this project is about clean data modelling and aggregation queries.

Models to build

- Category — name (e.g. Food, Rent, Salary), type (str: income/expense), owner_id
- Transaction — amount (float), description, date (date), category_id (FK), owner_id (FK)

Endpoints to implement

22. GET /transactions/ — filter by date range (?from=2024-01-01&to=2024-12-31), category, type
23. POST /transactions/ — create a transaction
24. PUT /transactions/{id} — update (owner only)
25. DELETE /transactions/{id} — delete (owner only)
26. GET /categories/ — list user's categories
27. POST /categories/ — create a category
28. GET /summary — return total income, total expenses, and balance for a date range

New concepts you will practice

- Aggregate queries: use SQLAlchemy's func.sum() to compute totals without loading every row into Python
- Date range filtering: `Transaction.date >= from_date` combined with `Transaction.date <= to_date`
- Computed response schemas: the /summary endpoint returns a custom schema (not a model row) — build a SummaryRead Pydantic model with total_income, total_expenses, balance
- No file uploads — good practice for a clean JSON-only API

Stretch goals
Add GET /summary/by-category — breakdown of spending per category for a period.
Add a monthly report endpoint: GET /summary/monthly?year=2024 returns 12 rows.
Add budget limits to categories and warn when a category exceeds its budget.

Suggested Progression Path

Stage	What to focus on
Start with Project 1 or 2	Get comfortable with the full CRUD loop in a new domain. Copy the BookLibrary structure exactly — new models, new schemas, same patterns.
Add the stretch goals	Each stretch goal introduces one new concept. Do them one at a time and make sure you understand why before moving on.
Try Project 3 or 4	These introduce more complex queries and multi-user ownership logic — the tricky part of real APIs.
Add tests	For every project, write at least: one test for the happy path, one for unauthorized access, one for a 404. Use the test_books.py file as your template.

Stage	What to focus on
Swap the database	Change DATABASE_URL to a PostgreSQL connection string and install asyncpg. Almost nothing in your code changes — that is the point of an ORM.

Tips for every project
1. Draw your models first. Before touching code, sketch the tables and their relationships.
Which fields are required? Which are optional? What are the FKs?
2. Write your schemas before your routes. Decide what data comes IN and what goes OUT
before you think about the HTTP layer.
3. Build CRUD functions before routes. Test them directly in a Python shell or a
simple script before wiring them to FastAPI.
4. Use Swagger UI (/api/v1/docs) as your test client while developing.
It's faster than Postman for quick iteration.
5. Read the error messages. FastAPI's 422 Unprocessable Entity responses include
a detailed JSON body explaining exactly which field failed and why.